multivariate linear regression

mahmood

May 8, 2023

note
this document and others can be found on my website
`https://mahmoodsheikh36.github.io/post/`

[?]
we can easily extend from univariate linear regression to **multivariate linear regression**, such that each example x_j is an n-element vector. our hypothesis space is the set of functions of the form

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1 x_{j,1} + \cdots + w_n x_{j,n} = w_0 + \sum_i w_i x_{j,i}$$

the w_0 term, the intercept, stands out as different from the others. we can fix that by inventing a dummy input attribute, $x_{j,0}$, which is defined as always equal to 1. then h is simply the dot product of the weights and the input vector, or equivalently the matrix product of the transpose of the weights and hte input vector:

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^{\mathrm{T}} \mathbf{x}_j = \sum_i w_i x_{j,i}$$

the best vector of weights, $\mathbf{w}^*$ minimizes squared-error loss over the examples:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j)$$

here we have a continuous weight space and we cannot find a closed-form solution. we use a hill-climbing algorithm that follows the gradient of the function. in this case, because we are trying to minimize the loss, we will use gradient descent. we choose any starting point in weight space–here, a point

in the (w_0, w_1) plane–and then move to a neighboring point that is downhill, repeating until we converge on the minimum possible loss, recall that the sum is minimized when the partial derivatives with respect to each of the weights are zero

```
w ← any point in the parameter space
repeat
    for w_i ∈ w do
        w_i ← w_i − α ∂/∂w_i Loss(w)
until convergence
```

the parameter α (the step size), is usually called the learning rate when we are trying to minimize loss in a learning problem. it can be a fixed constant, or it can decay over time as the learning process proceeds.
here the update equation for each weight w_i is

$$w_i \leftarrow w_i + \alpha \sum_j x_{j,i}(y_j - h_{\mathbf{w}}(\mathbf{x}_j))$$

in this equation we're considering not only the derivative of the loss function with respect to the weight we're updating but the derivatives of all of the loss functions with respect to each weight (unlike in the pseudocode above), i think this might be better but im not sure, i should evaluate both methods
it is also possible to solve analytically for the $\mathbf{w}$ that minimizes loss. let $\mathbf{y}$ be the vector of outputs for the training examples, and $\mathbf{X}$ be the data matrix, i.e., the matrix of inputs with one n-dimensional example per row. then the solution

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

minimizes the squared error.

note

im confused a little here, if we can solve for it analyitcally then why would we consider gradient descent in the first place (solving analytically requires less computing time)?
turns out, when fitting higher degree functions its not analytically solvable

with univariate linear regression we didnt have to worry about overfitting. but with multivariate linear regression in high-dimensional spaces it is possible that some dimension that is actually irrelevant appears by chance to be useful, resulting in overfitting.
thus, it is common to use **regularization** on multivariate linear functions to avoid overfitting. recall that with regularization we minimize the total cost of a hypothesis, counting both the empirical loss and the complexity of the hypothesis:

$$Cost(h) = EmpLoss(h) + \lambda \, Complexity(h)$$

for linear functions the complexity can be specified as a function of the weights. we can consider a family of regularization functions:

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

as with loss functions, with $q = 1$, we have L_1 regularization, which minimizes the sum of the absolute values; with $q = 2$, L_2 regularization minimizes the sum of squares. which regularization function should you pick? that depends on the specific problem, but L_1 regularization has an important advantage: it tends to produce a **sparse model**. that is, it often sets many weights to zero, effictively declaring the corresponding attributes to be irrelevant–just as Decision-Tree-Learning does (although by a different mechanism).hypothesis that discard attributes can be easier for a human to understand, and may be less likely to overfit

1 fitting higher order functions

if we wanted to fit a given data set to a higher order function, say quadratic, we just introduce n new weights and apply them to $x_{j,1}^2 \ldots x_{j,n}^2$ and add these new symbols to the hypothesis set equation

2 lisp implementation

enough talk, its time to implement this in common lisp
first we need the data

```
(defun generate-points-along-line (n)
  "Generate n points along a line with some random noise."
  "written by chatgpt, modified a little"
  (let ((slope 2)           ; slope of the line
        (intercept 1)      ; y-intercept of the line
        (noise-variance 5) ; variance of the noise
        (points '()))      ; list of points
    (dotimes (i n)
      (let* ((x (+ i 1))                ; x-coordinate of point
             (y (+ (* slope x) intercept)) ; y-coordinate of
                point on line
             (noise (/ (random (* 1000 noise-variance)) 1000)))
                ; random noise
        (push (list x (+ y noise)) points))) ; add point to
           list
    points))

(generate-points-along-line 5)
```

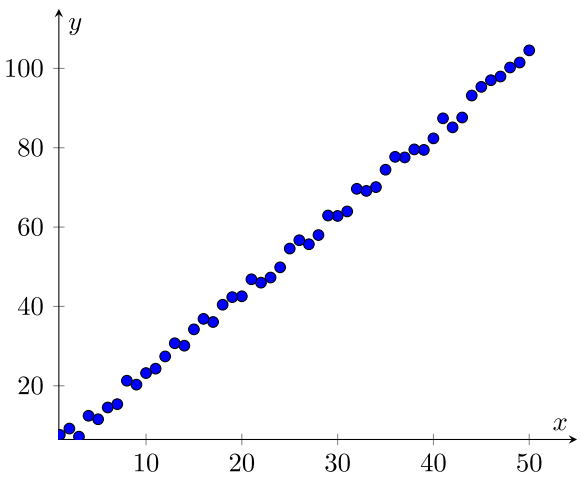
5	2731/200
4	10017/1000
3	9907/1000
2	5381/1000
1	381/100

we need a way to visualize this so we use tikz and org mode

```
(defun tikz-points-plot (points)
  "turn points into a runnable org mode latex block"
  (format t "#+begin_src latex :file (temp-file \"svg\") :
     exports results~%\\begin{tikzpicture}")
  (let ((max-x (car (car points)))
        (min-x (car (car points)))
        (max-y (car (car points)))
        (min-y (car (car points))))
    (loop for point in points
          do (cond ((> (car point) max-x) (setf max-x (car point
             ))))
             (cond ((< (car point) min-x) (setf min-x (car point
                ))))
             (cond ((> (nth 1 point) max-y) (setf max-y (nth 1
```

```
                point))))
            (cond ((< (nth 1 point) min-y) (setf min-y (nth 1
                point)))))
    (format t "\\begin{axis}[axis lines=middle, xlabel=$x$,
       ylabel=$y$, xmin=~a, xmax=~a, ymin=~a, ymax=~a]"
           (cond ((< min-x 0) (* 1.1 min-x))
                 ((> min-x 0) (* 0.9 min-x))
                 ((equal 0 min-x) -1))
           (cond ((< max-x 0) (* 0.9 max-x))
                 ((> max-x 0) (* 1.1 max-x))
                 ((equal 0 max-x) 1))
           (cond ((< min-y 0) (* 1.1 min-y))
                 ((> min-y 0) (* 0.9 min-y))
                 ((equal 0 min-y) -1))
           (cond ((< max-y 0) (* 0.9 max-y))
                 ((> max-y 0) (* 1.1 max-y))
                 ((equal 0 max-y) 1))))
  (loop for point in points
        do (format t "\\draw[fill=blue] (axis cs: ~a,~a) circle
            (2pt);" (car point) (nth 1 point)))
  (format t "\\end{axis}\\end{tikzpicture}~%#+end_src"))

(let ((points (generate-points-along-line 50)))
  (tikz-points-plot points))
```



i decided not to go with random data for now though... maybe later
i got a small dataset from `https://www.scribbr.com/statistics/multiple-linear-regressi`
the goal is to find the rates of heart disease as a function of biking to work

and smoking, so we have 3 weights to adjust (never forget the intercept!)

```
(defun loss (actual-output desired-output)
  (expt (- actual-output desired-output) 2))

(defun generate-hypothesis (data)
  (let* ((hypothesis-degree (length (car data)))
         (weights (loop for i from 0 below hypothesis-degree
            collect (/ (random 100) 100))))
    (values
     (lambda (vector) (reduce #'+ (mapcar #'* (append '(1)
        vector) weights)))
     weights)))

(defun predict (hypothesis vector)
  (funcall hypothesis vector))

(multiple-value-bind (hypothesis weights) (generate-hypothesis
    '((1 2 3) (2 3 4)))
  (print (predict hypothesis '(1 1))))
```

then we gotta load the dataset using cl-csv

```
(ql:quickload "cl-csv")
(defparameter *data* (cl-csv:read-csv (pathname data-file)))
(setf *data* (mapcar #'cdr *data*)) ;; get rid of the first
   column, its useless
(defun parse-float (mystr) ;; hacky way..
  (read-from-string mystr))
(setf *data* (mapcar (lambda (mylist) (mapcar #'parse-float
   mylist)) (cdr *data*)))
```

now we implement the gradient descent

```
(defun predict-all (data hypothesis weights)
  (let ((total-loss 0))
    (loop for vector in data
          do (setf total-loss (+ (predict hypothesis vector)
             total-loss))
             (let ((desired-output (car vector))
```

```
                  (x-values (cdr vector))
                  (this-examples-loss 0))
              (setf this-examples-loss (loss desired-output (
                 predict hypothesis x-values)))
              (setf total-loss (+ total-loss this-examples-loss
                 ))))
    total-loss))

(setq *learning-rate* 0.000000005)
(multiple-value-setq (hypothesis weights) (generate-hypothesis
   *data*))
(predict-all *data* hypothesis weights)

(defun step-down-gradient-descent (data weights)
  (let ((old-weights (copy-list weights))
        (counter 0))
    (map-into
     weights
     (lambda (weight)
       (let ((change-in-weight 0))
         (loop for example in data
               do (let ((desired-output (car example))
                        (actual-output (reduce #'+ (mapcar #'* (
                           append '(1) (cdr example)) old-weights
                           )))
                        (my-x (nth counter example)))
                    ;; (format t "~a * (~a - ~a)~%" my-x
                       desired-output actual-output)
                    (incf change-in-weight (* my-x (-
                       desired-output actual-output))))
                  (incf weight (* change-in-weight *
                     learning-rate*)))
         (incf counter))
       weight)
     weights)))

;; (loop for i from 0 below 10
;;       do (step-down-gradient-descent *data* weights)
;;          (format t "weights: ~a, loss: ~a~%" weights (
   predict-all *data* hypothesis weights)))
```

```
;; (loop for i from 0 below 100000
;;       do (step-down-gradient-descent *data* weights))
(loop for i from 0 below 100000
      do (step-down-gradient-descent *data* weights))
(format t "weights: ~a, loss: ~a~%" weights (predict-all *data*
    hypothesis weights))
```

```
weights: (15.316911 -0.2039037 0.17000248), loss: 10013.191
```

the function we got is very close to the actual function which is (according to the article i lined) `heart disease = 15 + (-0.2*biking) + (0.178*smoking)`

note

messing with the learning rate dramatically affected the final output, took me a bit of time to find a good value
if we had higher float decision we would've gotten better results but after some amount of iterations the loss just stops changing and converges to a specific number
the loss might seem like a high number at first but its really just because the values themselves in the dataset arent small

note

after messing with the learning-rate value, i realized how huge of an effect it had on the behavior of the algorithm, a slightly bigger step size causes the loss and weights to diverge for some reason and i cant see why